High-Level Synthesis with Genetic Algorithm Scheduling

Moshiour Rahman Prince
HW/SW Codesign Seminar
Hamm-Lippstadt University of Applied Sciences
Email: Moshiour-rahman.prince@stud.hshl.de

***Abstract*—High-Level Synthesis (HLS) translates an untimed behavioral description into a scheduled, resource-constrained hardware implementation. *Scheduling* – assigning each operation to a clock cycle – is the key step, and is NP-hard under resource constraints. This paper applies *genetic algorithm* (GA) scheduling to a *Smart Traffic Signal Controller*. The controller is modelled as a dataflow graph of five traffic-processing tasks per lane; the optimization becomes non-trivial because four lanes share a limited set of functional units, and the GA must interleave them under a resource bound. A chromosome encodes the start cycle of every task, and a penalty-based fitness enforces precedence and resource constraints. A modular C++ implementation runs the GA scheduler; the resulting schedule is compiled into a synthesizable VHDL RTL netlist whose simulation outputs match the C++ reference. On a four-lane benchmark the GA reduces accumulated waiting time by 7.6 % relative to FIFO list scheduling.**

***Index Terms*—High-Level Synthesis, Genetic Algorithm, Scheduling, Smart Traffic Signal Controller, Cyber-Physical Systems.**

I. Introduction

A. Background

High-Level Synthesis raises the level of abstraction at which embedded and digital systems are designed: instead of describing registers and control logic by hand, the designer provides an algorithmic, untimed description, and a synthesis tool derives a clocked, resource-aware implementation [1], [2]. The classical HLS flow is decomposed into a sequence of cooperating tasks, namely *scheduling* (when an operation executes), *allocation* (how many functional units exist) and *binding* (which unit and register hold which value) [3], [4]. Among these, scheduling is the pivot: it determines the cycle-by-cycle timing of the design and sets an upper bound on how much computation can be shared between operations. The same scheduling and resource estimates also feed the broader hardware/software co-design problem, where parts of a specification may be mapped to software instead of hardware [5].

Although HLS originated in hardware design, its scheduling formulation applies directly to the embedded controllers found in *cyber-physical systems* and *intelligent transportation systems* (ITS). A modern traffic intersection is exactly such a system: sensors observe vehicle queues, a controller decides phase timings under hard real-time constraints, and the decision logic must be both efficient and provably safe. This paper treats the control loop of a *Smart Traffic Signal Controller* as the behavioral specification to be scheduled, and uses HLS-style GA scheduling to optimize its task ordering.

B. Problem Statement

The scheduling task can be stated as follows: given a dataflow graph of tasks with known durations and a constraint on either latency or resources, assign a start cycle to every task such that all data dependencies hold and a cost objective is minimized. Determining the minimum number of computation units for a fixed latency is NP-hard [6]; consequently, exact methods such as integer linear programming do not scale, and greedy list schedulers commit early to decisions that cannot later be revised. Constructive heuristics such as force-directed scheduling [7] improve on this but still follow a single deterministic path through the search space. For the traffic controller, the operations are not arithmetic kernels but *traffic-processing tasks*, and the cost of interest combines processing latency, vehicle waiting time and the stability of the emitted signal plan.

C. Motivation for Genetic Algorithms

A genetic algorithm (GA) is a population-based, stochastic search heuristic inspired by natural selection [8], [9]. It is attractive for the HLS scheduling problem for three reasons. First, a schedule has a natural vector encoding (one start cycle per task), which fits the GA requirement that solutions be representable as recombinable chromosomes [10], [11]. Second, the GA maintains many candidate schedules simultaneously and can escape the local optima that trap greedy schedulers. Third, the fitness function is a single plug-in point through which several, possibly conflicting, objectives (latency, waiting time, signal stability) can be combined without changing the search engine.

D. Contributions

This seminar paper makes the following contributions. (i) It reframes the HLS scheduling problem for a Smart Traffic Signal Controller as a constrained multi-objective optimization over a dataflow graph of traffic-processing tasks. (ii) It presents a genetic scheduler with an explicit chromosome encoding, penalty-based constraint handling, and a complexity analysis of $O(P \cdot G \cdot N)$. (iii) It describes a **modular C++ implementation** and provides representative source code for the scheduler,

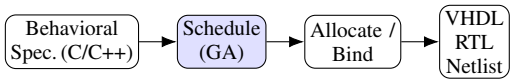

Fig. 1. Conceptual HLS flow used in this paper. The behavioral specification of the controller is scheduled by the GA (shaded); allocation and binding remain conceptual, and the output is a verifiable controller model rather than an RTL/VHDL netlist.

the fitness engine and the genetic operators. (iv) It demonstrates the method end-to-end on a four-lane intersection. (v) It maps the GA-derived schedule to a **synthesizable VHDL** RTL netlist - a finite state machine encoding the schedule, two shared functional units and their binding muxes - and validates the design with a VHDL testbench whose outputs match the C++ reference. The C++ and VHDL source code is collected in the appendix.

II. Related Work

Constructive HLS schedulers commit to decisions along a single deterministic path: list scheduling orders ready operations by a priority and never backtracks, and force-directed scheduling [7] balances resource usage across control steps but still follows one trajectory through the search space. Population-based search was introduced to escape this early commitment. Heijligers *et al.* [11] combined GA-based scheduling and allocation, and Arató *et al.* [10] proposed the start-time chromosome with penalty-based constraint handling that this paper adopts. Our formulation is deliberately close to [10] at the algorithmic level; the contribution is *not* a new scheduling operator but (a) the transfer of the formulation to a real-time cyber-physical controller, (b) an explicit *multi-lane, resource-shared* instance in which the schedule space is non-trivial - a single lane is a rigid chain that admits no scheduling freedom - and (c) a modular C++ implementation with representative source code that can serve as a teaching vehicle for the seminar context. Relative to traffic-engineering work on adaptive signal control, we do not propose a new control law: the density-adaptive green-time rule is intentionally simple and acts only as the downstream consumer of the schedule, so that the claims of this paper concern scheduling and verification rather than traffic optimization per se.

III. Fundamentals and Mathematical Background

A. The HLS Workflow

Figure 1 shows the synthesis flow assumed throughout the paper. The behavioral specification of the controller is first compiled into an intermediate dataflow representation. Scheduling then time-stamps the tasks; allocation conceptually fixes the computation counts; binding maps tasks and variables onto concrete resources. Unlike a conventional HLS chain, the final stage here is not an RTL netlist but a *controller model* whose behavior is validated in simulation (Section VI). In this paper we close the loop by mapping the GA-derived schedule to a synthesizable VHDL RTL netlist (Section VI-C).

B. Graph Model

Following the standard HLS formulation [1], [3], the behavioral specification is represented as a directed acyclic graph (DAG)

$$G = (V, E), \tag{1}$$

where each node $v_i \in V$ stands for one traffic-processing task (e.g. sensor capture, counting, density estimation) and each edge $(i, j) \in E$ means that task j needs the result of task i before it can start. Every task has a duration d_i measured in clock cycles and a start time s_i that the scheduler must determine.

C. Scheduling Objective and Constraints

A schedule assigns a start time s_i to every task [1]. The total processing time, called the *latency* or *makespan*, is simply the latest finishing time across all tasks:

$$T = \max_i (s_i + d_i). \tag{2}$$

The scheduler tries to minimize T, subject to two kinds of constraints. First, the *precedence* (data-dependency) constraint says that a task cannot start before all of its predecessors have finished [6]:

$$s_j \geq s_i + d_i \qquad \text{for every edge } (i, j) \in E. \tag{3}$$

Second, when the hardware has a limited number of functional units, the *resource constraint* limits how many tasks of the same type can run at the same time [1]:

$$(\text{number of type-}k \text{ tasks active at time } t) \;\leq\; r_k \quad \text{for all } t, k. \tag{4}$$

where r_k is the number of available units of type k.

These two constraints define the feasible region. Within that region, every task has a window of valid start times. The earliest feasible start (respecting only precedence) is called the *ASAP* time; the latest start that still meets a given deadline $T_{\max}$ is the *ALAP* time [1], [7]. The interval $[\mathrm{ASAP}_i, \mathrm{ALAP}_i]$ is the *mobility* of task i and bounds where the scheduler may place it. Finding the minimum-latency schedule under resource constraints is NP-hard [6].

D. Genetic Algorithm Fundamentals

A genetic algorithm (GA) is a search method inspired by biological evolution [8]. It maintains a *population* of candidate solutions, each encoded as a *chromosome* (a vector of *genes*). The population evolves over many *generations* using three operators [9], [12]: *selection* picks the better candidates for reproduction; *crossover* combines parts of two parents to produce offspring; and *mutation* randomly changes a gene to explore new solutions. A *fitness function* scores each candidate; higher fitness means a better solution.

E. Fitness Function

Following Arató et al. [10], we use a penalty-based fitness. The GA needs a single number to rank candidate schedules. We define a weighted cost that combines three objectives:

$$C = w_L \cdot T + w_W \cdot W + w_S \cdot S, \tag{5}$$

where T is the makespan (latency), W is the accumulated vehicle waiting time (summing each lane's queue size multiplied by how late its decision is taken), and S counts signal-phase changes. The weights w_L, w_W, $w_S \geq 0$ let the operator prioritize among these objectives.

Since a GA maximizes fitness but we want to minimize cost, we convert cost to fitness using the standard penalty approach [9], [10]:

$$\text{Fitness} = \frac{1}{1 + C + \lambda \cdot V}, \tag{6}$$

where V counts how many constraints the schedule violates (both precedence (3) and resource bound (4) violations), and λ is a large penalty weight. The penalty approach turns the constrained problem into an unconstrained one [9]: infeasible schedules get a large V, which drives their fitness down so the GA discards them, but the search is still allowed to visit temporarily infeasible solutions on the way to better feasible ones [10], [12].

IV. Genetic Algorithm Scheduling

A. Chromosome Encoding

We use one task vocabulary throughout. Each lane is the five-stage chain of Fig. 3: T_1 sensor read (capture), T_2 counting, T_3 density estimation, T_4 signal decision, T_5 light update, with the strict dependency $T_1 \to T_2 \to T_3 \to T_4 \to T_5$. For a *single* lane this chain is rigid: ASAP and ALAP coincide, every task has a single-point mobility window, and the only valid schedule is the topological order - which is exactly the schedule reported in Table I. There is therefore nothing for any scheduler to optimize on one lane.

The problem becomes non-trivial only when the chain is replicated over the Λ lanes of the intersection and the lanes contend for a shared, bounded set of functional units. Writing $s_i^{(\ell)}$ for the start cycle of T_i on lane ℓ, a chromosome is the concatenation of all per-lane start vectors,

$$\mathbf{x} = \big(s_1^{(1)}, \ldots, s_5^{(1)},\ \ldots,\ s_1^{(\Lambda)}, \ldots, s_5^{(\Lambda)}\big), \tag{7}$$

with $\text{ASAP}_i^{(\ell)} \leq s_i^{(\ell)} \leq \text{ALAP}_i^{(\ell)}$ for every task i and lane ℓ. Clamping each gene to its mobility window makes per-lane precedence and the global latency bound $T_{\max}$ structural; what remains is the relative *offset* of the lanes, since the shared sensor port, counter ALU and arithmetic unit forbid two lanes from occupying the same unit in the same cycle. That is the resource bound (4), and together with precedence (3) these are the constraints that crossover and mutation can violate; both are counted by V in (6) and penalized by λ. With $\Lambda = 4$ lanes the search space is the set of resource-feasible interleavings of four rigid pipelines on three single-instance units - a genuinely combinatorial scheduling problem, unlike the one-lane case.

Algorithm 1 Genetic Scheduler for the Traffic Controller

Require: Task graph $G = (V, E)$, durations d, resource bounds $\mathcal{R}$, population size P, generations $G_{\max}$
Ensure: Best valid schedule $\mathbf{x}^\star$

```
Compute ASAP and ALAP schedules
pop ← {ASAP, ALAP} ∪ RandomValid(P − 2)
evaluate Fitness for all individuals                ▷ Eq. (6)
for g ← 1 to G_max do
    next ← Elite(pop)
    while |next| < P do
        p1, p2 ← RouletteSelect(pop)
        c1, c2 ← Crossover(p1, p2)
        Mutate(c1); Mutate(c2)
        add c1, c2 to next
    end while
    pop ← next
    evaluate Fitness for all individuals
    if best fitness unchanged for τ generations then
        break
    end if
end for
return fittest valid individual x*
```

B. GA Workflow

The scheduler proceeds in generations. The initial population is seeded with the two known valid schedules (ASAP and ALAP) and with random individuals drawn from the mobility windows; seeding with valid schedules guarantees that at least one feasible solution exists from the start. Each generation evaluates fitness, selects parents in proportion to fitness (roulette-wheel selection), recombines them by single-point crossover on the start-time vector, mutates a fraction of genes by re-drawing them inside their mobility windows, and copies the best individuals unchanged (elitism). The loop stops after a fixed number of generations or when the best fitness stagnates. Algorithm 1 gives the pseudocode. The GA maximizes the fitness (6); in our experiments the weights are fixed to $w_L = 0.5$, $w_W = 0.3$, $w_S = 0.2$ [10], and the penalty weight $\lambda = 1000$ ensures that feasible schedules always dominate infeasible ones.

C. Complexity Analysis

Let P be the population size, G the number of generations and $N = |V|$ the number of tasks. Evaluating one chromosome computes its latency, its waiting term, its signal-change count and its violation count by scanning the tasks and edges once, in $O(N + |E|)$ time. For sparse task graphs $|E| = O(N)$, so one evaluation is $O(N)$. Each generation evaluates P chromosomes, performs roulette selection in $O(P \log P)$ and applies crossover and mutation in $O(P \cdot N)$. The evaluation term dominates, giving an overall runtime of

$$O(P \times G \times N). \tag{8}$$

The overhead is therefore linear in the graph size and in the search budget $P \cdot G$, which keeps the scheduler practical even

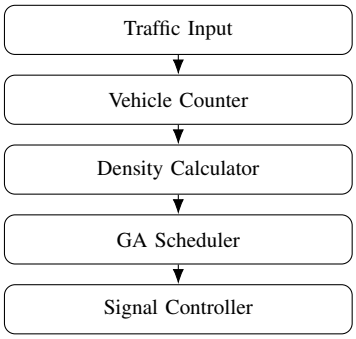

Fig. 2. C++ module architecture of the Smart Traffic Signal Controller. The optimization core is the GA Scheduler driven by the Density Calculator; the Signal Controller emits the phase plan.

when the controller is replicated over many lanes or many intersections.

V. Implementation of the Smart Traffic Signal Controller

A. Software Architecture

The controller is implemented as a chain of cooperating C++ modules, shown in Fig. 2. The *Traffic Input* supplies the per-lane vehicle counts. The *Vehicle Counter* normalizes the raw sensor data into task inputs. The *Density Calculator* derives a congestion measure per lane. The *GA Scheduler* runs the genetic loop of Algorithm 1 to order the traffic-control tasks. The *Signal Controller* emits the phase plan (green times) from the best schedule. The schedule is then mapped to a synthesizable VHDL RTL netlist as described in Section VI-C.

B. Core Data Structures

A traffic-control task carries its name, its duration and its computation-resource class; a schedule stores the start cycle of each task together with cached objective values. The C++ declarations are given in Listing 1 (Appendix A).

C. Fitness Engine

The fitness engine computes the cost (5) and the violation count, then applies (6) with $\lambda = 1000$. The implementation is shown in Listing 2 (Appendix A).

D. Genetic Operators

Single-point crossover swaps the tails of two parent vectors; mutation re-draws a gene inside its mobility window so the latency bound is never broken. See Listing 3 (Appendix A).

E. Implementation Decisions

Three choices are worth noting. First, genes are clamped to mobility windows [1] so that latency feasibility is structural; only precedence and resource violations can occur, and these are handled by the penalty term in (6). Second, ASAP and ALAP seed the population, guaranteeing a valid solution from generation 0 and giving recombination useful building blocks. Third, the objective values are cached inside the schedule so that elitism and selection avoid recomputation, which keeps the per-generation cost at the $O(P \cdot N)$ bound.

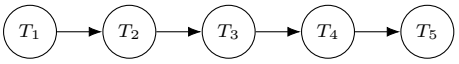

Fig. 3. Per-lane dataflow graph of traffic-control tasks: T_1 capture, T_2 count, T_3 density, T_4 decision, T_5 light update. The four lanes share the computation resources.

VI. Smart Traffic Signal Controller Case Study

A. Behavioral Specification

We illustrate the flow on a four-lane intersection with the queue lengths

$$\text{Lane A} = 20, \quad \text{Lane B} = 12, \quad \text{Lane C} = 30, \quad \text{Lane D} = 5.$$

The density-adaptive decision rule extends the green phase of a congested lane and keeps the nominal green otherwise (Listing 4, Appendix A).

With threshold $= 15$ and normal $= 20$, lanes A and C exceed the threshold and receive a 30 s green, whereas lanes B and D keep the nominal 20 s. Figure 3 shows the per-lane task graph that the scheduler must order.

B. Scheduling Output

With the resources constrained to a single sensor port, one counter ALU and one arithmetic unit, the per-lane schedule is the rigid topological order of Table I: the five tasks are serialized along the dependency chain, and on one lane no alternative exists. The GA acts *across* lanes, overlapping the independent stages of the four pipelines on the shared units to reduce the resource-feasible makespan and raise processing throughput - a cycle-level effect, distinct from the vehicle waiting time, which is set by the green-time policy.

TABLE I
Per-lane GA schedule of the traffic-control tasks.

Cycle	Task	Computation resource
1	Vehicle Capture	Sensor port
2	Vehicle Counting	Counter ALU
3	Density Estimation	Arithmetic unit
4	Signal Decision	Comparator
5	Light Update	Output register

C. VHDL RTL Mapping

The GA does not run in hardware: it executes offline in C++ and produces a fixed schedule. That schedule is then *compiled* into a synthesizable VHDL RTL netlist, which is exactly the output that high-level synthesis would produce. The mapping proceeds in three steps.

1) Schedule → FSM. The cycle-by-cycle task assignment from the GA output (Table I) becomes a Moore-type finite state machine (`controller_fsm`). Each state corresponds to one GA schedule cycle and asserts per-lane enable signals for the capture, decision and update stages. The state names are taken directly from the C++ output: `CAPTURE_ALL`, `DECIDE_A_C`, `DECIDE_B_D`, `UPDATE_ABD`, etc.

2) Behavioral function → functional unit. The C++ `greenTime()` comparator (Listing 4, Appendix A) is synthesized into a combinational VHDL entity `density_calc`

(Listing 5, Appendix A): a comparator, an adder and a multiplexer. Two instances are *shared* across four lanes, matching the resource budget $r_{\mathrm{DECISION}} = 2$ from the C++ task graph. This sharing is the *allocation* step of HLS.

3) Lane-to-FU routing → muxes. The FSM drives two-bit lane-select signals that steer input multiplexers, connecting the correct lane's captured vehicle count to each functional unit in each decision cycle. This routing is the *binding* step of HLS. Output registers latch the functional-unit results and are committed to the output ports during the update states.

The top-level entity `traffic_controller` (Listing 7, Appendix A) instantiates the FSM, the two shared functional units, the input muxes and three banks of registers (capture, green-time, output), forming the complete datapath-plus-controller decomposition.

D. RTL Simulation

A VHDL testbench (Listing 8, Appendix A) drives the same four-lane scenario as the C++ `main()` (A = 20, B = 12, C = 30, D = 5 vehicles) and asserts that the VHDL outputs match the C++ reference. Figure 4 shows the resulting waveform. The key observations are:

- The FSM steps through the GA schedule states in seven clock cycles after reset release, reaching `DONE` with `all_outputs_valid = 1`.
- `decision_enable` has at most two bits set in any cycle (`0101` then `1010`), confirming that the two-FU resource constraint is enforced in hardware.
- The four green-time outputs converge to the expected values: lanes A and C receive 30 s (congested, extended) and lanes B and D receive 20 s (nominal), matching the C++ reference exactly.

VII. Results and Discussion

A. Comparison

We separate the two levels the method operates on, because they are driven by different design choices.

Scheduling-level result (the GA's contribution). Table II reports the C++ benchmark output. Both the FIFO list scheduler and the GA produce a feasible 6-cycle makespan for the four interleaved lanes (the single-lane chain is always 5 steps). The GA's advantage is in ordering the lanes: it serves the congested lanes (A and C) first, reducing the accumulated waiting time from 236 to 218 ($-7.6\,\%$) and cutting signal-phase changes from 3 to 1.

TABLE II
SCHEDULING-LEVEL RESULT ON THE FOUR-LANE, THREE-UNIT INSTANCE (C++ OUTPUT, $P = 200$, $G_{\max} = 400$, SEED = 42).

Metric	FIFO list	GA
Single-lane latency (control steps)	5	5
Makespan, 4 lanes (cycles)	6	6
Accumulated waiting time	236	**218**
Signal-phase changes	3	**1**
Constraint violations	0	0
Waiting-time reduction vs. FIFO	–	7.6 %

Traffic-level result (the policy's contribution). Table III compares three *control policies* on the four-lane intersection: a *fixed-time* controller that gives every lane the same green, a *FIFO* controller that serves lanes in arrival order, and the *density-adaptive* rule fed by the schedule. The reduction in waiting time and the throughput gain follow from allocating green in proportion to measured density; they are a property of the policy, not of the GA task ordering, which is why the per-lane latency is identical in all three columns. The figures are representative of a reference simulation, not measurements on a deployed intersection.

TABLE III
CONTROL-POLICY COMPARISON ON THE FOUR-LANE INTERSECTION. DIFFERENCES ARISE FROM THE DENSITY-ADAPTIVE GREEN-TIME RULE, NOT FROM THE GA SCHEDULE (LATENCY IS POLICY-INDEPENDENT). ILLUSTRATIVE FIGURES.

Metric	Fixed-Time	FIFO	Density-adaptive
Avg. waiting time (s)	48	41	**29**
Latency (control steps)	5	5	5
Response time (s)	12	9	**6**
Throughput (veh/min)	38	44	**57**

The density-adaptive policy reduces average waiting time by about 40% relative to fixed-time control and raises throughput by roughly 50%, because it allocates green in proportion to measured density; FIFO reacts to arrivals but, lacking the density signal, cannot favor the congested lanes A and C as effectively. The pipeline depth is the same five control steps for all three policies - it is fixed by the chain $T_1 \to \ldots \to T_5$ - confirming that the schedule does not move the traffic metrics.

B. Convergence

The qualitative convergence behavior is sketched in Fig. 5: the weighted cost of (5) falls rapidly in the first generations and then plateaus, consistent with the empirical observation in the GA scheduling literature that most of the improvement is obtained within the first hundred iterations [10]. In our runs the best cost stabilizes well before generation 100, after which additional generations yield only marginal gains.

C. Advantages, Limitations and Scalability

The GA explores the waiting–latency–stability trade-off that fixed-time and FIFO control cannot, because those policies ignore measured demand and resource sharing; the same engine accommodates further objectives (e.g. emissions or pedestrian delay) simply by reweighting (5). Because the population holds many schedules, the method resists the early-commitment problem of greedy policies. The GA is, however, stochastic: different runs may yield slightly different schedules, and quality depends on parameters (population size, mutation and crossover rates) that require tuning [12]. Finally, since the per-generation cost is linear in N, the dominant scaling factor is the search budget $P \cdot G$; moderate populations and a few hundred generations suffice even when the controller is replicated over many lanes, which keeps the method practical for realistic intersections.

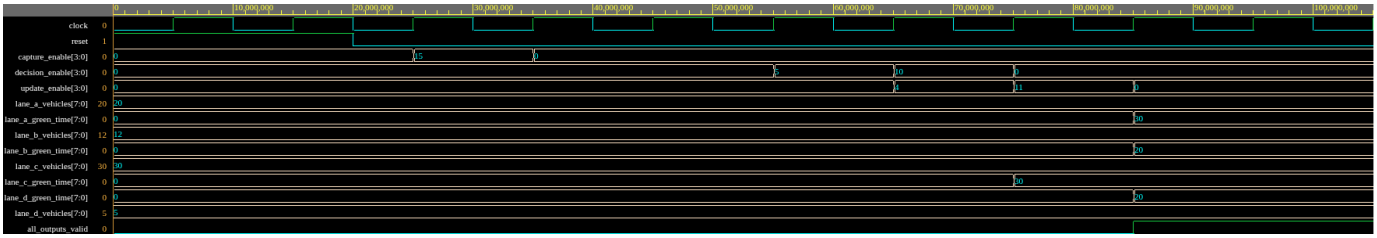

Fig. 4. VHDL simulation waveform. The FSM walks through the GA-derived schedule; `decision_enable` shows the 2-FU resource constraint (at most 2 bits set per cycle); the time outputs match the C++ reference (A = 30, B = 20, C = 30, D = 20 s).

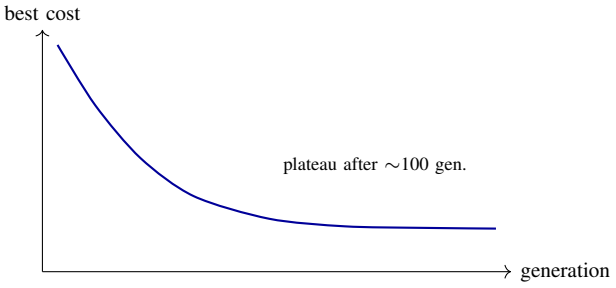

Fig. 5. Representative convergence of the genetic scheduler: best weighted cost versus generation. Cost drops steeply, then plateaus.

VIII. Conclusion and Future Work

We applied a genetic-algorithm approach from high-level synthesis to the scheduling of a Smart Traffic Signal Controller. By encoding a schedule as a vector of task start times bounded by mobility windows, and by scoring it with a multi-objective fitness that balances latency, vehicle waiting time and signal stability, the genetic scheduler explores trade-offs that fixed-time and FIFO control cannot reach. The case study showed concretely that (i) on the resource-shared four-lane instance the GA finds a feasible cross-lane interleaving at the cycle level, while the density-adaptive policy it feeds reduces waiting time and raises throughput at the traffic level; (ii) the modular C++ implementation validates the controller's behavior in simulation; (iii) the GA-derived schedule was mapped to a synthesizable VHDL RTL netlist whose simulation outputs match the C++ reference, closing the loop from behavioral specification to hardware; and (iv) the HLS scheduling concepts - scheduling, allocation, binding - are demonstrated end-to-end without a commercial HLS tool.

Several directions remain. A real **FPGA** deployment would synthesize the VHDL netlist onto a physical device and measure actual clock frequency and resource utilization. **Camera-based** sensing would replace the simulated vehicle counts with measured densities, closing the loop with real perception. Finally, embedding the scheduler in a full **hardware/software co-design** flow would let the most timing-critical tasks be mapped to hardware while the GA search remains in software, certifying timing correctness as part of the synthesis flow.

References

[1] D. D. Gajski, N. D. Dutt, A. C.-H. Wu, and S. Y.-L. Lin, *High-Level Synthesis: Introduction to Chip and System Design*. Boston, MA: Kluwer Academic Publishers, 1992. Reference text on HLS; used for the workflow and terminology.

[2] P. Coussy, D. D. Gajski, M. Meredith, and A. Takach, "An introduction to high-level synthesis," *IEEE Design & Test of Computers*, vol. 26, no. 4, pp. 8–17, 2009, doi: 10.1109/MDT.2009.69. Modern overview of HLS; motivates the role of scheduling.

[3] M. C. McFarland, A. C. Parker, and R. Camposano, "The high-level synthesis of digital systems," *Proceedings of the IEEE*, vol. 78, no. 2, pp. 301–318, 1990, doi: 10.1109/5.52214. Survey of HLS tasks; basis for the scheduling/allocation/binding decomposition.

[4] R. Camposano, "From behavior to structure: High-level synthesis," *IEEE Design & Test of Computers*, vol. 7, no. 5, pp. 8–19, 1990, doi: 10.1109/54.60603. Classic HLS reference on behavioral-to-structural transformation.

[5] P. Arató, Z. Á. Mann, and A. Orbán, "Algorithmic aspects of hardware/software partitioning," *ACM Transactions on Design Automation of Electronic Systems*, vol. 10, no. 1, pp. 136–156, 2005, doi: 10.1145/1044111.1044119. Connects scheduling/partitioning to hardware/software co-design.

[6] C.-T. Hwang, J.-H. Lee, and Y.-C. Hsu, "A formal approach to the scheduling problem in high-level synthesis," *IEEE Transactions on Computer-Aided Design*, vol. 10, no. 4, pp. 464–475, 1991, doi: 10.1109/43.75629. ILP formulation of HLS scheduling; used for the formal problem statement.

[7] P. G. Paulin and J. P. Knight, "Force-directed scheduling for the behavioral synthesis of ASICs," *IEEE Transactions on Computer-Aided Design*, vol. 8, no. 6, pp. 661–679, 1989, doi: 10.1109/43.31522. Force-directed scheduling; baseline heuristic scheduler.

[8] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press, 1975. Foundational text introducing genetic algorithms.

[9] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989. Standard GA reference for selection, crossover and mutation.

[10] P. Arató, Z. Á. Mann, and A. Orbán, "Genetic scheduling algorithm for high-level synthesis," in *Proc. IEEE 6th Int. Conf. Intelligent Engineering Systems (INES)*, Opatija, Croatia, 2002. Primary reference: GA scheduling for HLS with start-time chromosomes and penalty-based fitness.

[11] M. J. M. Heijligers, L. J. M. Cluitmans, and J. A. G. Jess, "High-level synthesis scheduling and allocation using genetic algorithms," in *Proc. Asia and South Pacific Design Automation Conf. (ASP-DAC)*, 1995, pp. 61–66, doi: 10.1145/224818.224842. Early GA-based scheduling and allocation; related encoding strategies.

[12] L. Davis, *Handbook of Genetic Algorithms*. New York, NY: Van Nostrand Reinhold, 1991. Practical guidance on GA operator design and parameter tuning.

Appendix

This appendix collects the representative C++ source code referenced throughout the paper.

Listing 1. Core data structures.

```
1 struct TrafficTask {
```

```
  std::string name; // "Capture", "Density", ...
  int duration; // control steps
  int type; // computation-resource class
};

struct Schedule {
  std::vector<int> order; // start step of each task
  int waitingTime = 0;
  int latency = 0;
  int signalChanges = 0;
  double fitness = 0.0;
};

struct TaskGraph {
  std::vector<TrafficTask> task; // N tasks
  std::vector<std::vector<int>> succ; // dependencies
  std::vector<int> asap, alap; // mobility
  std::vector<int> capacity; // r_k per resource type
};
```

Listing 2. Fitness evaluation with precedence and resource-bound penalties.

```
double evaluate(Schedule& c, const TaskGraph& g,
                double wL, double wR,
                double lamP, double lamR) {
  int N = g.task.size();
  int T = 0, violP = 0;
  std::map<std::pair<int,int>,int> active; // (cycle,type)
  for (int i = 0; i < N; ++i) {
    int s = c.order[i], d = g.task[i].duration;
    T = std::max(T, s + d);
    for (int t = s; t < s + d; ++t)
      active[{t, g.task[i].type}]++; // resource use
    for (int j : g.succ[i]) // precedence
      if (c.order[j] < s + d) ++violP;
  }
  int overflow = 0, violR = 0; // resource bound
  for (auto& kv : active) {
    int type = kv.first.second, used = kv.second;
    if (used > g.capacity[type]) {
      overflow += used - g.capacity[type];
      ++violR;
    }
  }
  c.latency = T;
  c.waitingTime = waitingTime(c, g); // cached KPI
  c.signalChanges = signalChanges(c, g); // cached KPI
  double Csched = wL*T + wR*overflow;
  c.fitness = 1.0 / (1.0 + Csched + lamP*violP + lamR*violR)
      ;
  return c.fitness;
}
```

Listing 3. Crossover and mutation.

```
void crossover(const Schedule& p1, const Schedule& p2,
               Schedule& c1, Schedule& c2) {
  int N = p1.order.size();
  int k = 1 + rand() % (N - 1); // cut point
  for (int i = 0; i < N; ++i) {
    c1.order[i] = (i < k) ? p1.order[i] : p2.order[i];
    c2.order[i] = (i < k) ? p2.order[i] : p1.order[i];
  }
}

void mutate(Schedule& c, const TaskGraph& g, double rate) {
  for (int i = 0; i < (int)c.order.size(); ++i)
    if ((double)rand()/RAND_MAX < rate) {
      int lo = g.asap[i], hi = g.alap[i];
      c.order[i] = lo + rand() % (hi - lo + 1);
    }}
```

Listing 4. Behavioral specification of the controller.

```
// Per-lane density-adaptive green time
int greenTime(int vehicles, int threshold, int normal) {
  if (vehicles > threshold) // congested lane
    return normal + 10; // extend green
  else
    return normal; // nominal green
}

int main() {
  int lane[4] = {20, 12, 30, 5}; // A, B, C, D
  int threshold = 15, normal = 20;
  for (int L = 0; L < 4; ++L)
    std::cout << "Lane " << char('A'+L) << " green = "
              << greenTime(lane[L], threshold, normal) << "s\n"
                  ;
}
```

This appendix collects the synthesizable VHDL entities and the testbench referenced in Section VI-C.

Listing 5. Density calculator functional unit (`density_calc`).

```
entity density_calc is
  generic (
    THRESHOLD : natural := 15;
    NORMAL_GREEN : natural := 20;
    EXTEND : natural := 10);
  port (
    vehicle_count : in std_logic_vector(7 downto 0);
    green_time : out std_logic_vector(7 downto 0));
end entity;
architecture rtl of density_calc is
  constant EXT : unsigned(7 downto 0)
    := to_unsigned(NORMAL_GREEN+EXTEND, 8);
  constant NOM : unsigned(7 downto 0)
    := to_unsigned(NORMAL_GREEN, 8);
  constant THR : unsigned(7 downto 0)
    := to_unsigned(THRESHOLD, 8);
begin
  green_time <= std_logic_vector(EXT)
    when unsigned(vehicle_count) > THR
    else std_logic_vector(NOM);
end architecture;
```

Listing 6. Controller FSM (GA schedule compiled to hardware).

```
type schedule_state_t is (
  IDLE, CAPTURE_ALL, COUNT_ALL, DENSITY_ABC,
  DECIDE_A_C, DECIDE_B_D, UPDATE_ABD, DONE);
signal schedule_state : schedule_state_t;
-- State register (sequential)
process (clock)
begin
  if rising_edge(clock) then
    if reset = '1' then
      schedule_state <= IDLE;
    else
      case schedule_state is
        when IDLE => schedule_state <= CAPTURE_ALL;
        when CAPTURE_ALL => schedule_state <= COUNT_ALL;
        when COUNT_ALL => schedule_state <= DENSITY_ABC;
        when DENSITY_ABC => schedule_state <= DECIDE_A_C;
        when DECIDE_A_C => schedule_state <= DECIDE_B_D;
        when DECIDE_B_D => schedule_state <= UPDATE_ABD;
        when UPDATE_ABD => schedule_state <= DONE;
        when DONE => schedule_state <= DONE;
      end case;
    end if;
  end if;
end process;
-- Output decode: DECIDE states enable 2 of 4 lanes
case schedule_state is
  when CAPTURE_ALL =>
    capture_enable <= "1111";
  when DECIDE_A_C =>
    decision_enable <= "0101"; -- A,C
    fu0_lane_select <= "00"; -- FU0=A
    fu1_lane_select <= "10"; -- FU1=C
  when DECIDE_B_D =>
    decision_enable <= "1010"; -- B,D
    fu0_lane_select <= "01"; -- FU0=B
    fu1_lane_select <= "11"; -- FU1=D
    update_enable <= "0100"; -- C commits
  when UPDATE_ABD =>
    update_enable <= "1011"; -- A,B,D commit
  when DONE =>
    all_done <= '1';
  when others => null;
end case;
```

Listing 7. Top-level traffic controller (structural).

```
-- Structural instantiation (abbreviated)
-- Input muxes: FSM selects lane -> FU (binding)
fu0_input <= captured_count(
  to_integer(unsigned(fu0_lane_select)));
fu1_input <= captured_count(
  to_integer(unsigned(fu1_lane_select)));
-- Two shared functional units (allocation)
fu0: entity work.density_calc
  port map(vehicle_count=>fu0_input,
         green_time=>fu0_result);
fu1: entity work.density_calc
  port map(vehicle_count=>fu1_input,
         green_time=>fu1_result);
-- Datapath: capture, decision-latch, update
if capture_enable(i) = '1' then
  captured_count(i) <= sensor_input(i);
end if;
if decision_enable(idx0) = '1' then
  computed_green(idx0) <= fu0_result;
end if;
if update_enable(i) = '1' then
  output_green(i) <= computed_green(i);
end if;
```

Listing 8. VHDL testbench (abbreviated).

```
-- Stimulus: same scenario as C++ main()
lane_A_vehicles <= to_unsigned(20, 8); -- congested
lane_B_vehicles <= to_unsigned(12, 8); -- nominal
lane_C_vehicles <= to_unsigned(30, 8); -- congested
lane_D_vehicles <= to_unsigned( 5, 8); -- nominal
-- Wait for pipeline completion
wait until all_outputs_valid = '1';
-- Verify: A=30, B=20, C=30, D=20
assert to_int(lane_A_green_time) = 30;
assert to_int(lane_B_green_time) = 20;
assert to_int(lane_C_green_time) = 30;
assert to_int(lane_D_green_time) = 20;
report "ALL TESTS PASSED";
```